

Task 01:

```

import sys

print("\n===== TASK 1 =====\n")

class MyClass:
    def __init__(self, x, y):
        self.x = x
        self.y = y

obj = MyClass(10, 20)
print("1. Namespace of instance:", obj.__dict__)

def student_data(student_id, student_name=None, student_class=None):
    print("\n2. Student Data:")
    print("Student ID:", student_id)
    if student_name:
        print("Student Name:", student_name)
    if student_class:
        print("Class:", student_class)

student_data(101)
student_data(102, "Hania", "10th")

class Student:
    pass

print("\n3. Class Inspection:")
print("Type:", type(Student))
print("Class __dict__ keys:", Student.__dict__.keys())
print("Module:", Student.__module__)

class Student:
    pass

class Marks:
    pass

s = Student()
m = Marks()

print("\n4. Instance and Inheritance Check:")
print("s is instance of Student:", isinstance(s, Student))
print("m is instance of Marks:", isinstance(m, Marks))
print("Student subclass of object:", isinstance(Student, object))
print("Marks subclass of object:", isinstance(Marks, object))

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

s = Student("Hania", 85)

print("\n5. Modify Attributes:")
print("Original:", s.name, s.marks)

s.name = "Nida"
s.marks = 90

print("Updated:", s.name, s.marks)

class Student:
    def __init__(self, student_id, student_name):
        self.student_id = student_id

```

```

        self.student_id = student_id
        self.student_name = student_name

s = Student(1, "Hania")

s.student_class = "10th"

print("\n6. Before deletion:", s.__dict__)

del s.student_name

print("After deletion:", s.__dict__)

class Student:
    def __init__(self, student_id, student_name, student_class):
        self.student_id = student_id
        self.student_name = student_name
        self.student_class = student_class

    def display(self):
        print("\n7. Display Attributes:")
        for key, value in self.__dict__.items():
            print(key, ":", value)

s = Student(1, "Hania", "10th")
s.display()

class Student:
    pass

student1 = Student()
student2 = Student()

student1.name = "Hania"
student1.id = 1

student2.name = "Nida"
student2.id = 2

print("\n8. Two Students Data:")
for i, student in enumerate([student1, student2], start=1):
    print(f"\nStudent {i}:")
    for key, value in student.__dict__.items():
        print(key, ":", value)

```

```

class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

s = Student("Hania", 85)

print("\n9. Memory Usage:")
print("Object size:", sys.getsizeof(s))
print("Dict size:", sys.getsizeof(s.__dict__))

class Student:
    __slots__ = ['name', 'marks']

    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

s = Student("Hania", 85)

print("\n10. Memory with __slots__:")
print("Object size:", sys.getsizeof(s))

```

===== TASK 1 =====

1. Namespace of instance: {'x': 10, 'y': 20}

```

2. Student Data:
Student ID: 101

2. Student Data:
Student ID: 102
Student Name: Hania
Class: 10th

3. Class Inspection:
Type: <class 'type'>
Class __dict__ keys: dict_keys(['__module__', '__dict__', '__weakref__', '__doc__'])
Module: __main__

4. Instance and Inheritance Check:
s is instance of Student: True
m is instance of Marks: True
Student subclass of object: True
Marks subclass of object: True

5. Modify Attributes:
Original: Hania 85
Updated: Nida 90

6. Before deletion: {'student_id': 1, 'student_name': 'Hania', 'student_class': '10th'}
After deletion: {'student_id': 1, 'student_class': '10th'}

7. Display Attributes:
student_id : 1
student_name : Hania
student_class : 10th

8. Two Students Data:

Student 1:
name : Hania
id : 1

Student 2:
name : Nida
id : 2

9. Memory Usage:
Object size: 48
Dict size: 296

10. Memory with __slots__:
Object size: 48

```

Task 02:

```

print("\n\n==== TASK 2 =====\n")

class Trapezoid:
    def __init__(self, a, b, h):
        self.a = a
        self.b = b
        self.h = h

    def area(self):
        return 0.5 * (self.a + self.b) * self.h

class Parallelogram:
    def __init__(self, base, height):
        self.base = base
        self.height = height

    def area(self):
        return self.base * self.height

results = {}

data_sets = [
    (10, 8, 5, 6, 4),
    (12, 6, 7, 5, 3),
    (15, 10, 8, 9, 6)
]

```

```
for i, (a, b, h, base, height) in enumerate(data_sets, start=1):
    trap = Trapezoid(a, b, h)
    para = Parallelogram(base, height)

    trap_area = trap.area()
    para_area = para.area()

    results[f"Set {i}"] = {
        "Trapezoid": trap_area,
        "Parallelogram": para_area
    }
```

```
print("Areas for different sets:")
for key, value in results.items():
    print(f"\n{key}:")
    for shape, area in value.items():
        print(f"{shape} Area = {area}")
```

```
max_shape = ""
max_area = 0
```

```
for dataset in results.values():
    for shape, area in dataset.items():
        if area > max_area:
            max_area = area
            max_shape = shape
```

```
print("\nShape with maximum area:", max_shape)
print("Maximum area:", max_area)
```

```
===== TASK 2 =====
```

```
Areas for different sets:
```

```
Set 1:
Trapezoid Area = 45.0
Parallelogram Area = 24
```

```
Set 2:
Trapezoid Area = 63.0
Parallelogram Area = 15
```

```
Set 3:
Trapezoid Area = 100.0
Parallelogram Area = 54
```

```
Shape with maximum area: Trapezoid
Maximum area: 100.0
```